

Sessions, Endpoint Intelligence & QA Report — Deployment Verification

📅 Sep 26, 2026 · @Pellumi

Three workstreams landed in one commit: correctness and tenancy fixes across the Sessions and Endpoint Intelligence surfaces, a versioned ClickHouse metrics table that the QA report now quotes a baseline from, and a frontend rollup plus five finding rules bringing the browser track to parity with the backend track. 41 files, ~5,130 insertions.

Read the first two sections before deploying anything — the work is not on the branch you may expect, and two new environment variables must be set or service-to-service calls will start returning 401 in production.

Where the code is

The work is committed as `a6787b8` on `dev`. It is not on the feature branch, and that branch is now stale.

I created `feat/sessions-endpoints-qa-report` and worked there, but left the tree uncommitted when I hit the usage limit. While I was paused, the work was committed onto `dev` and the branch pointer was left behind. `feat/sessions-endpoints-qa-report` still points at `6eb3506` — the commit the session *started* on — so it contains none of this work. Deploy from `dev`, or cherry-pick `a6787b8`; do not deploy the feature branch.

`dev` has also taken three commits from other sessions since mine:

Commit	What it is
<code>a6787b8</code>	This work — 41 files, all three workstreams
<code>b351999</code>	Another session (<code>jsssssjs</code>)
<code>94afa75</code>	Merge of <code>claude/inspiring-antonelli-5525ef</code>
<code>4642b04</code>	Another session, current <code>dev</code> tip

Those three are not mine and I have not reviewed them. Notably one of them adds a Prisma migration (`20260926120000_dashboard_finding_truth`) and touches `packages/rules`, the api-gateway and coverage-engine. **Anything you deploy from `dev` carries their changes too**, so treat this document as covering `a6787b8` only.

To see exactly what is mine:

```
git show --stat a6787b8
```

I re-ran my test suites against the current `dev` tip after those merges and they still pass, so the merges did not break this work.

Before you deploy

Two new environment variables — required in production

Variable	Set on	Why
<code>REPORT_ENGINE_INTERNAL_SECRET</code>	<code>report-engine</code> , <code>demonstration-api</code>	demonstration-api calls <code>GET /reports/:id/latest</code> with no user token
<code>ENDPOINT_ENGINE_INTERNAL_SECRET</code>	<code>endpoint-engine</code> , <code>report-engine</code> , <code>background-workers</code>	report-engine and the report worker call endpoint-engine with no user token

Both sides of each pair need the **same value**. Any secret will do — generate with `openssl rand -hex 32`.

The failure mode if you skip this: report-engine and endpoint-engine now verify their caller. Outside production an unauthenticated call is treated as internal so the dev stack keeps working, but **in production it is rejected**. So with `NODE_ENV=production` and no secret set, demonstration results and endpoint intelligence start returning 401. This is the single most likely way this deploy breaks something.

Already added to `.env.example` and `turbo.json`'s `globalPassThroughEnv`. **Not** added to the Railway configs under `infrastructure/railway/` — those are per-service JSON and I did not want to guess at your secret management. That is the one manual step.

Two new npm dependencies

`jsonwebtoken@^9.0.2` plus `@types/jsonwebtoken` were added to `services/report-engine` and `services/endpoint-engine`, which verify JWTs locally now. `pnpm-lock.yaml` is updated in the commit, so a normal `pnpm install --frozen-lockfile` picks them up.

Prisma: no migration from this work

I made no schema changes. `packages/db/prisma/schema.prisma` is untouched by `a6787b8`, so nothing here needs `prisma migrate deploy`.

But another commit on `dev` added `20260926120000_dashboard_finding_truth`. If you deploy from `dev` you will need to run migrations for *that* work. Confirm with:

```
git show --stat a6787b8 -- packages/db/
```

Empty output confirms this work needs no migration.

ClickHouse: a new table appears at boot

endpoint-engine creates `endpoint_metrics_v2` on startup, the same way it always created `endpoint_metrics`. No migration runner, no manual DDL.

The old `endpoint_metrics` table is left in place and is now **unread and unwritten**. Nothing reads it after this deploy, so endpoint intelligence starts from an empty history and fills as traffic arrives. That is expected — the old table could not be reused because its grouping key was the concrete path rather than the route template. Dropping it is a manual follow-up whenever you are comfortable.

Deploy order

`packages/shared` is built first by turbo regardless, but if you deploy services independently: **shared → endpoint-engine → report-engine → background-workers → session-engine → clickhouse-ingester → demonstration-api**, then the dashboard and desktop.

The one real ordering constraint: clickhouse-ingester must not be the first to boot after this change, because it no longer creates `endpoint_metrics` — endpoint-engine now owns that data. In practice both tables already exist, so this is only a concern on a fresh ClickHouse.

What changed per service

`packages/shared` — new exports

`ERROR_EVENT_TYPES`, `ERROR_EVENT_TYPE_SET`, `isErrorEventType`, `readErrorEventDetail`, `canonicalRoute`, `canonicalRouteFromPath`. Pure additions; nothing removed.

`ERROR_EVENT_TYPES` covers `ERROR_EVENT`, `ERROR_OCCURRED`, `UNHANDLED_EXCEPTION`, `SERVER_ERROR`, `CLIENT_ERROR`. `WORKFLOW_FAILED` is deliberately excluded — a workflow that did not complete is a flow outcome, not a runtime error, and including it would inflate every error count in the product.

session-engine — error counts and restart durability

- Statistics count every error-shaped event, not just `ERROR_EVENT`.
- `complete()` now claims by `create` on the unique `sessionId` and returns whether it won. Only the winner uploads the replay and emits `SESSIONS_COMPLETED`, so a re-completion cannot double-announce.
- A 60-second sweeper finds sessions with no statistics whose `endTime` is older than the idle timeout plus grace, and completes them. This is query-driven, so it recovers sessions that a restart orphaned — previously those were lost permanently.
- `SIGTERM` stops consuming, drains pending timers (10s cap), then disconnects.

After deploy: expect a burst of log lines like `Swept N orphaned session(s)` on first boot as it clears the existing backlog. Historical sessions still show — until you run the backfill (see the checklist).

report-engine — tenancy, environment scoping, error normalisation

New `src/auth.ts`. These routes now require a caller:

```
/reports/:applicationId/latest, /applications/:id/graph,
/applications/:id/workflows, /applications/:id/sessions,
/sessions/:sessionId/replay, /qa-runs/:runId/report,
/reports/:applicationId/endpoint-intelligence,
/reports/:applicationId/export
```

`/health` stays open. The guard accepts a bearer JWT, then an `access_token` cookie, then the internal secret. It never trusts an inbound `x-tellann-*` header.

Also: session listing is environment-scoped (defaults to the application's default environment) and returns `applicationTotal` and `environmentId`; the replay `errors` array covers all error types and reads the message through the shared normaliser; `?notifyEmail=true` on the export GET is gone, replaced by `POST /reports/:applicationId/export/notify`.

endpoint-engine — new table, bounded queries, new routes

New `src/auth.ts`, same guard. Routes:

Route	Status
GET /endpoints/:id/analysis	guarded; now grouped by route template, windowed, env-scoped
GET /endpoints/:id/baseline	new — what the report quotes
POST /endpoints/:id/alerts/notify	new — replaces ?notifyEmail=true
DELETE /endpoints/:id/retention	new — called by the retention sweeper

The analysis response gained `windowDays`, `requestedWindowDays`, `windowClampedByRetention`, `sampleCount`, `environmentId`, and each row gained `route` and `p50Ms`. Thresholds moved from mean to p95 and picked up the QA report's "3× its own median" rule, so the two systems now agree on what "slow" means. Minimum volume of 5 calls now applies to the slow ranking too, not just error-prone.

Window defaults to 30 days, clamps to 90, and is further clamped to the organisation's plan `retentionDays` — so a 14-day plan sees 14 days and the response says why.

`clickhouse-ingester` — stopped competing for a table

Its `endpoint_metrics` DDL and insert path are removed; it creates and writes only `events`. That path was almost certainly already dead: it looked for `endpoint` at the top level or under `payload`, while a `TellannEvent` carries it under `metadata`.

`background-workers` — report generation

The report payload gains `sections.frontendSummary`, `sections.backendSummary.endpoints[].baseline`, `sections.evidenceAppendix.eventsByType`, and `coverage.confidence / confidenceBand / caveats`. `summaryText` is rewritten to cover frontend-only, backend-only, mixed and observational runs. `retention-worker` now also expires ClickHouse metrics per tenant.

The baseline lookup is strictly additive — a 5-second timeout, and any failure degrades to a `baselineStatus` string while leaving every measured number untouched. **A report can never fail because endpoint-engine is down.**

`demonstration-api` — one call now authenticated

Its call to `GET /reports/:id/latest` presents `REPORT_ENGINE_INTERNAL_SECRET`. This is the service most likely to break if that variable is missing.

apps/dashboard

Sessions list: pagination fixed for non-default page sizes, environment filter, split empty states. Endpoints page: 403 renders an upgrade prompt, 503 a banner keeping the last data, empty the activation state. QA run detail: session cards linking into the replay viewer, raw JSON behind a toggle.

apps/desktop

A "What the browser saw" chapter in the PDF and new CSV row types (Browser , Route , Resource , Accessibility , Console , Form , Coverage caveat). A FrontendReportCard on the report page. Five new browser finding rules in @tellann/browser-observer .

New finding categories

FRONTEND_POOR_WEB_VITAL , FRONTEND_PAGE_CRASH , FRONTEND_FAILED_RESOURCES , FRONTEND_FORM_VALIDATION_FAILED , FRONTEND_ROUTE_NEVER_SETTLED .

BrowserFinding.category is a free-form string in both the Zod schema and Postgres, so **no schema change was needed** — but any dashboard filter or alert rule that enumerates categories will not know about these until you add them.

The one thing that can break production

Eight previously-open routes now verify their caller. That is the point of the change, but it means anything calling them that I did not find will start getting 401 or 404.

What I found and fixed

I traced every caller of these routes in the monorepo. Three service-to-service calls had no user token, and all three now present the internal secret:

Caller	Target
demonstration-api	report-engine GET /reports/:id/latest
report-engine (2 call sites)	endpoint-engine GET /endpoints/:id/analysis
background-workers (new)	endpoint-engine GET /endpoints/:id/baseline

The desktop app sends Authorization: Bearer <jwt> and the dashboard sends the access_token cookie, which the Next rewrite and the gateway both forward. Both are covered by the bearer-then-cookie order.

What I could not check

Anything outside this repository. If you have a CI job, a cron, a monitoring probe, an internal tool or a Postman collection hitting `/reports/*`, `/sessions/*` or `/endpoints/*` directly, it will start failing. I have no way to see those from here — **this is the thing worth checking on your side before you deploy.**

Also worth knowing: `packages/e2e-tests` declares a `REPORT_ENGINE_URL` constant in two files but never uses it, so those suites are unaffected.

A deliberate behaviour choice

A resource the caller may not see returns **404, not 403**. That matches the convention already used throughout `desktop-routes.ts` — confirming that another tenant's run exists is itself a small leak. If a monitoring probe distinguishes 403 from 404, it will see 404.

The dev-mode fallback, stated plainly

When the internal secret is unset and `NODE_ENV !== 'production'`, an unauthenticated request is treated as internal and allowed through, with a one-time warning logged. That keeps the local stack working without secrets.

It also means **a staging environment that does not set `NODE_ENV=production` is effectively unguarded on these routes.** If staging holds real tenant data, set both the secret and `NODE_ENV=production` there. I would treat this as the second thing to verify.

Verification checklist

Ordered so a failure stops you before it costs anything. Substitute your own ids for `<app>`, `<org-b-app>`, `<session>`, `<run>`.

1. Tenancy — do this first

The security fix is the reason this branch exists, so prove it works before trusting anything else.

- ☐ Signed in as a member of org A, request an org B application's sessions: `curl -i -b "access_token=<org-A-token>" "$GATEWAY/applications/<org-b-app>/sessions"` → expect **404**, not data
- ☐ Same for `/sessions/<org-b-session>/replay`, `/endpoints/<org-b-app>/analysis`, `/qa-runs/<org-b-run>/report` → **404** each
- ☐ With no cookie and no bearer at all → **401**

- ☐ With a forged `x-tellann-user-id: <some-uuid>` header and nothing else → **401** (the header is never trusted)
- ☐ Signed in as a member of org A, request your **own** application's sessions → **200** with data
- ☐ Open a completed QA run in the desktop app → the report still loads (proves the bearer path)

2. Service-to-service

- ☐ `GET /demonstrations/<id>/results` → **200**, not 401 (proves `REPORT_ENGINE_INTERNAL_SECRET` is set on both sides)
- ☐ Endpoints page in the dashboard loads → proves report-engine can reach endpoint-engine
- ☐ Generate a QA report on a backend-track run and check the worker logs for `baselineStatus` — `READY:30d` means the baseline lookup works; `NOT_CONFIGURED` or `UNAVAILABLE:*` means `ENDPOINT_ENGINE_INTERNAL_SECRET` or `ENDPOINT_ENGINE_URL` is missing on background-workers. **The report still generates either way** — this is a warning, not a failure.

3. Error counting

- ☐ Drive an application that emits `UNHANDLED_EXCEPTION` or `SERVER_ERROR` (not `ERROR_EVENT`)
- ☐ Sessions list shows a non-zero Errors column → previously showed 0
- ☐ Open the replay → the Errors panel is populated and each row shows its event type
- ☐ Dry-run the backfill, then run it: `node scripts/backfill-session-error-counts.mjs --dry-run` then without the flag. It is idempotent and re-runnable, and only writes rows whose count actually differs.

4. Session durability

- ☐ Start a session, kill session-engine mid-session, restart it
- ☐ Within ~90 seconds the session gains statistics, a replay object appears in object storage, and a `SESSIONS_COMPLETED` message is produced
- ☐ Logs show `Swept N orphaned session(s)`
- ☐ Confirm **no duplicate** `SESSIONS_COMPLETED` for the same `sessionId` — the claim-by-create is what prevents this

5. ClickHouse

- ☐ `SHOW TABLES` → `endpoint_metrics_v2` exists

- ☐ `SELECT route, endpoint, run_id, environment_id FROM endpoint_metrics_v2 LIMIT 20` → `route` is a **template** (`/orders/{param}`), not a concrete path (`/orders/17`). This is the cardinality fix; if you see concrete paths, the SDK is not sending `metadata.route` and the fallback is collapsing by shape instead — still correct, worth noting.
- ☐ During a QA run, rows carry a non-empty `run_id`
- ☐ Only endpoint-engine logs a table-ready line — clickhouse-ingester should no longer mention `endpoint_metrics`
- ☐ `GET /endpoints/<app>/analysis?windowDays=7` → response echoes `windowDays: 7`
- ☐ On a plan with 14-day retention, ask for 90 → response has `windowDays: 14` and `windowClampedByRetention: true`

6. Endpoints page states

All three should look different from each other:

- ☐ Plan without `ENDPOINT_INTELLIGENCE` → upgrade prompt linking to billing (**not** "connect the SDK")
- ☐ endpoint-engine stopped → amber banner saying the analysis service is unreachable
- ☐ Fresh application with no traffic → the activation empty state

7. QA run → session link

- ☐ Open a completed run in the dashboard → "Sessions this run observed" shows cards, not a JSON dump
- ☐ Click one → lands on the right session replay with the right `appId`
- ☐ "Show raw correlation" still reveals the JSON

8. The QA report itself

Run a **frontend-track** run against a deliberately poor page — an oversized hero image, a layout shift, a 404'd font, a form that fails validation, a route that polls forever:

- ☐ `sections.frontendSummary` is populated with real percentiles
- ☐ All five finding rules fire, once per route each
- ☐ Desktop report page shows the "What the browser saw" card
- ☐ PDF has the browser chapter; CSV has `Browser / Route / Resource / Accessibility / Console / Form` rows
- ☐ A metric the browser could not measure reads **"Not measured"**, never `0`

Then:

- ☐ **Backend-only run** → no browser card and no browser chapter at all (the null return, not an empty section)
- ☐ **Mixed-track run** → `summaryText` carries both a browser clause and a server clause
- ☐ **Degraded capture** (production environment, or force a `CAPTURE_DEGRADED` finding) → `coverage.confidenceBand` drops and `coverage.caveats` lists the reasons; the PDF gains a `CONFIDENCE` row and a "What qualifies that number" block
- ☐ **A run exceeding 2,000 evidence events** → every finding-linked event is still in the appendix, `eventsByType` is published, and events remain chronological

9. Regression check on the backend section

- ☐ An application with **more than 100 distinct routes** → the endpoint table caps at 100, but `requests`, `p95Ms` and `slowestRequests` count all of them, and the limitation text says "100 of N routes". Before this change the totals silently excluded everything past the cap.

Failures that are not regressions

Four things fail in this repo that are not caused by this work. Listed so you do not spend time on them.

What fails	Why it is not mine
<code>apps/desktop/src/renderer/pages.tsx</code> — Property <code>'icon'</code> is missing (around line 10152)	I verified this by stashing all my changes and re-running the renderer typecheck against the clean base tree. The error is present there too.
<code>@tellann/shared</code> lint	The package has no <code>eslint.config.js</code> , and ESLint 9 requires one. It has never had a working lint task.
<code>@tellann/e2e-tests</code>	Every failure is <code>ECONNREFUSED</code> against <code>localhost:3006</code> and friends. These are integration tests needing a live stack; they pass nothing without one.
<code>@tellann/db</code> build — <code>EPERM ... rename query_engine-windows.dll.node</code>	A Windows file lock. A running dev server or a stray test process holds the Prisma engine DLL. It cleared as soon as I stopped a leftover process of my own. Not an issue on Linux.

Test results as of the current `dev` tip

Re-run after the other sessions' merges landed:

Package	Tests	New in this work
<code>packages/shared</code>	26 pass	+13
<code>services/background-workers</code>	70 pass	+45
<code>packages/browser-observer</code>	49 pass	+9
<code>apps/desktop</code> (main)	84 pass	+11

`pnpm build` passes across the monorepo.

One thing worth knowing about the build

Running the build regenerated `apps/docs/src/generated/legacy-redirects.ts` with different content than what is committed — the committed version contains `http://localhost:3020` destinations, so whoever last generated it built with a local

marketing URL. I reverted my regeneration to keep the diff clean, but **the committed artifact points at localhost**, which is probably wrong for production docs redirects. Separate issue, worth a look.

Two test scripts changed shape

`services/background-workers` and `apps/desktop` used to enumerate their test files by name, so a new test file silently never ran. Both now glob:

```
node --test "dist/**/*.test.js"
```

If your CI invokes those scripts, nothing changes. If it enumerates files itself, it will miss the new tests.

Deliberately not done

Four items from the plan were left out. None of them block this deploy.

AI resolutions for frontend findings. Backend findings get an AI-drafted resolution with real code context; frontend ones still do not. This needs `packages/ai`'s bundle type split into a shared core plus two arms. The honest scope is also narrower than it sounds: only network and HTTP findings carry usable code context, because a `BrowserFinding` holds no source location. Console errors point at a bundled asset and source maps are not captured, so they could only get route-level context. Accessibility findings could get none — a bare CSS selector is too thin a signal.

Cross-track causal chains. `interactionGroupId`, `causedByEventId` and `traceId` already link a click to its requests to the server request that served it, and the payload still does not assemble them. This is the most distinctive thing the data model can express and the report does not use it.

Run-over-run comparison. `retryOfRunId` exists on `QARun` and nothing reads it. The ClickHouse baseline that did land answers "is this route normally slow"; this would answer "did my fix work". It needs an index on that column.

Two further spec'd rules. `docs/app/frontend-qa-capture.md` also requires detecting "interaction with no expected effect" and "high-confidence absence of pagination". Both are real gaps and outside what was asked for here.

Worth noting: the five rules that *did* land were not new scope either — slow render timings, broken pages, and form validation failures are all listed in that same spec as required deterministic analysis, and were simply unimplemented.

Rollback

The change is mostly additive, which makes rollback cheap.

Code: `git revert a6787b8`. Clean unless the later commits on `dev` touched the same lines.

Environment: the new variables become inert. Leave them.

ClickHouse: `endpoint_metrics_v2` stays behind, harmless. A reverted clickhouse-ingester resumes creating and writing `endpoint_metrics`, and a reverted endpoint-engine resumes reading it — so the old path restores itself, but with a gap in its history for however long the new table was live.

Database: nothing to undo. No schema change, and the only data written is `SessionStatistic.errorCount` values that are now correct — a revert leaves them correct while the code goes back to undercounting.

Partial rollback is safe. If only the tenancy guards are a problem, reverting just `services/report-engine/src/auth.ts` and `services/endpoint-engine/src/auth.ts` plus their route wiring restores open access without touching the report or ClickHouse work.

If something needs following up

The working plan with full design rationale is at `~/.claude/plans/pasted-content-id-9144-okay-we-greedy-shannon.md` on my side, including the per-item reasoning and the ordering constraints. Ask and I can bring any of it into this doc.